

Project 2: A Bite of Distributed Communication

Project 2: A Bite of Distributed Communication

Zophia Kimberly Laud [029783219],

Ivy Le [030215976],

Mai Mai [030276114]

Instructor: Hailu Xu

California State University, Long Beach

College of Engineering

CECS 327, Sec 02, Spring 2026

March 20, 2026

Links:

Video - <https://youtu.be/DoAr8xY807g>

Task 1: Anycast with Docker (TCP)

For the Anycast task, we implemented a simulated “Anycast” system where multiple servers would listen on the same port and a client connects to just a single server instance, which is determined by Docker’s network load balancing.

We developed a TCP server in `server.py` using Python’s `socket` module. Each server was paired with a unique identifier and configured with each other, demonstrating which instance handled what request.

We then used a `docker-compose.yml` file to launch three server containers and one client container. All servers listened on port 5000, and Docker’s network balanced the client requests. We verified the connection by running the client multiple times to show it connecting to different servers.

```
○ (base) ~mai@mai-book task1 % docker compose up
WARN[0000] No services to build
[+] up 5/5
  ✓ Network task1_appnet Created
  ✓ Container server3    Created
  ✓ Container server1    Created
  ✓ Container server2    Created
  ✓ Container client     Created
Attaching to client, server1, server2, server3
client | Received: Hello from server3
client exited with code 0
task1-client-run-dcd724985427 | Received: Hello from server3
task1-client-run-dcd724985427 exited with code 0
task1-client-run-acd08a6adfd5 | Received: Hello from server1
task1-client-run-acd08a6adfd5 exited with code 0
task1-client-run-2682a3f7c045 | Received: Hello from server2
task1-client-run-2682a3f7c045 exited with code 0
task1-client-run-6afe060db257 | Received: Hello from server3
task1-client-run-6afe060db257 exited with code 0
```

Project 2: A Bite of Distributed Communication

```
● (base) mai@mai-book task1 % docker compose run --rm client
WARN[0000] No services to build
[+] 3/3t 3/33
✓ Container server2 Running
✓ Container server1 Running
✓ Container server3 Running
WARN[0000] No services to build
Container task1-client-run-6afe060db257 Creating
Container task1-client-run-6afe060db257 Created
Received: Hello from server3
○ (base) mai@mai-book task1 %
```

We also used tcpdump within the containers to capture the handshakes and data transfer, verifying each specific IP address.

```
docker exec -it server1 tcpdump -i eth0 tcp port 5000%
○ (base) mai@mai-book task1 % docker exec -it server1 tcpdump -i eth0 tcp port 5000
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
■
```

Task 2: Multicast with UDP

Docker was used to build a system where a sender broadcasts messages to multiple receivers in a multicast group. To do this, we developed `multicast_receiver.py`, which joins the group and remains active for a specific duration, specified using the `--duration` parameter flag. The `multicast_sender.py` file was configured to send both JSON data and binary data to the group.

We also added a second sender to observe the multiple sources in the traffic logs. We also used tcpdump inside the receiver container to capture the UDP packets and compare them to the program output.

Project 2: A Bite of Distributed Communication

```
(base) mai@mai-book task2 % docker compose up
WARN[0000] No services to build
Attaching to receiver1, receiver2, sender1, sender2
sender1 | Sent JSON: {'sensor': 'temp', 'value': 20.5}
sender1 | Sent binary: b' vd\x18A\xc0\xc9\xa9'
sender1 | Sent JSON: {'sensor': 'temp', 'value': 21.5}
sender1 | Sent binary: b'\x80\xe8\xd8\xe9S\xd3'
sender1 | Sent JSON: {'sensor': 'temp', 'value': 22.5}
sender1 | Sent binary: b'\xdf\xf0\xcfy\xc32\x94f'
sender1 | Sent JSON: {'sensor': 'temp', 'value': 23.5}
sender1 | Sent binary: b'\xc5h\xc9L\x0e\xa4\xcd\xcd'
sender1 | Sent JSON: {'sensor': 'temp', 'value': 24.5}
sender1 | Sent binary: b'\x19n\x88\xe1\x1d\xcab\x90'
sender2 | Sent JSON: {'sensor': 'humidity', 'value': 30.0}
sender2 | Sent binary: b'\x06\x94\xcb\xe50\x99}\xd4'
sender2 | Sent JSON: {'sensor': 'humidity', 'value': 31.0}
sender2 | Sent binary: b'\xb3D\xd0\x13\x7f\x89\xb9F'
sender2 | Sent JSON: {'sensor': 'humidity', 'value': 32.0}
sender2 | Sent binary: b'k\xa8\xb9\x11L\xe2\xe7\xf5'
sender2 | Sent JSON: {'sensor': 'humidity', 'value': 33.0}
sender2 | Sent binary: b'\xba\x9514\xf3\xa6\r\x19'
sender2 | Sent JSON: {'sensor': 'humidity', 'value': 34.0}
sender2 | Sent binary: b'\xe4/0\xdf\xaa\xdb\xef\xe9'
sender2 exited with code 0
sender1 exited with code 0
receiver2 | Joined multicast group 224.1.1.1:5007
receiver2 | Received JSON from ('172.19.0.4', 55763): {'sensor': 'humidity', 'value': 30.0}
receiver1 | Joined multicast group 224.1.1.1:5007
receiver2 | Received binary from ('172.19.0.4', 55763): b'\x06\x94\xcb\xe50\x99}\xd4'

receiver2 | Received JSON from ('172.19.0.5', 59578): {'sensor': 'temp', 'value': 20.5}
receiver1 | Received JSON from ('172.19.0.4', 55763): {'sensor': 'humidity', 'value': 30.0}

receiver2 | Received binary from ('172.19.0.5', 59578): b' vd\x18A\xc0\xc9\xa9'
receiver1 | Received binary from ('172.19.0.4', 55763): b'\x06\x94\xcb\xe50\x99}\xd4'
```

Project 2: A Bite of Distributed Communication

```
receiver2 | Received binary from ('172.19.0.5', 59578): b'\xdf\xf0\xcf\x32\x94f'
receiver1 | Received binary from ('172.19.0.4', 55763): b'k\xa8\xb9\x11L\xe2\xe7\xf5'

receiver2 | Received JSON from ('172.19.0.4', 55763): {'sensor': 'humidity', 'value': 33.0}
receiver1 | Received JSON from ('172.19.0.5', 59578): {'sensor': 'temp', 'value': 22.5}

receiver2 | Received binary from ('172.19.0.4', 55763): b'\xba\x9514\xf3\xa6\r\x19'
receiver1 | Received binary from ('172.19.0.5', 59578): b'\xdf\xf0\xcf\x32\x94f'
receiver1 | Received JSON from ('172.19.0.4', 55763): {'sensor': 'humidity', 'value': 33.0}
receiver2 | Received JSON from ('172.19.0.5', 59578): {'sensor': 'temp', 'value': 23.5}

receiver1 | Received binary from ('172.19.0.4', 55763): b'\xba\x9514\xf3\xa6\r\x19'
receiver2 | Received binary from ('172.19.0.5', 59578): b'\xc5h\xc9L\x0e\xa4\xcd\xcd'
receiver2 | Received JSON from ('172.19.0.4', 55763): {'sensor': 'humidity', 'value': 34.0}
receiver1 | Received JSON from ('172.19.0.5', 59578): {'sensor': 'temp', 'value': 23.5}
receiver2 | Received binary from ('172.19.0.4', 55763): b'\xe4/0\xdf\xaa\xdb\xef\xe9'

receiver1 | Received binary from ('172.19.0.5', 59578): b'\xc5h\xc9L\x0e\xa4\xcd\xcd'

receiver2 | Received JSON from ('172.19.0.5', 59578): {'sensor': 'temp', 'value': 24.5}
receiver1 | Received JSON from ('172.19.0.4', 55763): {'sensor': 'humidity', 'value': 34.0}

receiver2 | Received binary from ('172.19.0.5', 59578): b'\x19n\x88\xe1\x1d\xcab\x90'
receiver1 | Received binary from ('172.19.0.4', 55763): b'\xe4/0\xdf\xaa\xdb\xef\xe9'
receiver2 | Leaving multicast group
receiver1 | Received JSON from ('172.19.0.5', 59578): {'sensor': 'temp', 'value': 24.5}
receiver1 | Received binary from ('172.19.0.5', 59578): b'\x19n\x88\xe1\x1d\xcab\x90'
receiver1 | Leaving multicast group
receiver2 exited with code 0
receiver1 exited with code 0
(base) mai@mai-book task2 %
```

```
⊙ (base) mai@mai-book task2 % docker exec -it receiver1 tcpdump -i eth0 udp port 5007
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
21:14:20.370718 IP sender2.task2_mcastnet.55763 > 224.1.1.1.5007: UDP, length 37
21:14:20.370872 IP sender2.task2_mcastnet.55763 > 224.1.1.1.5007: UDP, length 8
21:14:20.372480 IP sender1.task2_mcastnet.59578 > 224.1.1.1.5007: UDP, length 33
21:14:20.372645 IP sender1.task2_mcastnet.59578 > 224.1.1.1.5007: UDP, length 8
21:14:22.376620 IP sender2.task2_mcastnet.55763 > 224.1.1.1.5007: UDP, length 37
21:14:22.376749 IP sender2.task2_mcastnet.55763 > 224.1.1.1.5007: UDP, length 8
21:14:22.378464 IP sender1.task2_mcastnet.59578 > 224.1.1.1.5007: UDP, length 33
21:14:22.378523 IP sender1.task2_mcastnet.59578 > 224.1.1.1.5007: UDP, length 8
⊙ (base) mai@mai-book task2 %
```

Contributions

Mai Mai - Anycast server/client implementation, tcpdump log analysis, and testing.

Project 2: A Bite of Distributed Communication

Ivy Le - Multicast sender/receiver implementation, JSON/Binary data handling, Docker Compose configuration.

Zophia Kimberly Laud - Multi-container testing, networking validation, report preparation.

Challenges

1. When configuring the Anycast simulation, we found that the client would consistently connect to the same server container instead of switching between different server instances.
 - a. We realized that it was because Docker's internal DNS caches the IPs, so we modified our client execution to make sure a fresh connection was established each time, so that the load balancer could actually pick between the containers.
2. We noticed that some packets were missing during the multicast task.
 - a. After some research, we realized this actually was because UDP is considered unreliable, whereas TCP has guaranteed delivery.

References

<https://docs.python.org/3/library/socket.html>

<https://docs.docker.com/compose/how-tos/networking/>